

Konkurentno programiranje

Katedra za automatsko upravljanje
Fakultet tehničkih nauka
Novi Sad

24. mart 2026.

Zašto konkurentno?

- konkurentno - više različitih akcija tokom nekog vremenskog intervala (*OS* rukovodi), paralelno - u istom vremenskom trenutku
- proces - pokrenut program, nit - nezavistan tok izvršavanja nekog skupa instrukcija; proces ima n niti i aktivan je dok ima bar jednu aktivnu nepozadinsku nit
- svaka nit ima svoj stek i prioritet, ID, funkciju koja izvršava i argumente koje prima; „*jefinije*” ju je stvoriti, preključiti se i uništiti je od procesa; sve niti u okviru jednog procesa dele memoriju
- bolja iskorišćenost hardvera
- brži proračun
- „*Free lunch is over*” - uvek postoji neki *trade-off*
- „*Some people, when confronted with a problem, think, 'I know, I'll use threads,' and then two they hav erpoblems.*”
- greške koje se javljaju su teške za detektovati i debugovati, ponekad se i nasumično javljaju (*Rust* jezik i *ThreadSanitizer* u *gcc*-u mogu pomoći)
- *Da li 2 niti znači duplo a 10 niti deset puta brže izvršavanje?*

Načini komunikacije

1 slanje poruka - radićemo u nastavku kursa

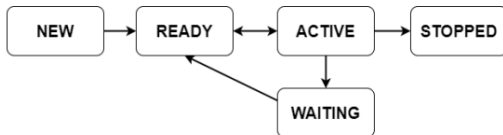
- one-way
- request-response
- duplex
- ★ sinhrono
- ★ asinhrono
- polling
- callback
- message queue
- publish-subscribe
- streaming (socket)

2 preko deljene memorije - ovim se bavimo do kraja ovog predavanja

- ne kopiraju se podaci
- sinhronizacija je nužna

- **Start, Join, detach, stop, yield**

Na slici ispod se nalaze konceptualno stanja u kojoj se nit može naći.



- *context switching* - set operacija koje *CPU* radi prilikom preključivanja niti (čuvanje stanja niti, kopiranje registara, određivanje koja nit je sledeća...)
- *scheduler* može prekinuti nit ukoliko je nit većeg prioriteta spremna ili ukoliko se jedna nit predugo izvršava (eng. *time slicing*)

Race condition

- *Race condition* je nekontrolisan pristup podacima i ishod operacija (ne mora biti netačan ishod svaki put kad pokrenemo).
- Benigni *race condition* - ne proizvodi grešku u programu, na primer ispisi nakon izračunavanja (sinhronizacija ovakvih operacija je suvišna i smanjuje performanse).
- Trka za podacima (eng. *data race*) je situacija u kojoj više niti pristupa memorijskoj lokaciji konkurentno i pritom je bar jedna operacija pisanje i nije izvršena sinhronizacija. Ovakav tip problema se možda neće odmah javiti i zavisice i od hardvera na kom se izvršava, ostalih procesa itd; kompajler ne pomaže, čak i *unit* testovi mogu proći.

Atomičnost

Koje sve vrednosti **x** može imati nakon pozivanja obe funkcije ako se operacija ***=** deli na: čitanje, množenje i upis i možemo se preključiti u bilo kom trenutku?

```
-
static int x = 1;
0 references
static void Foo()
{
    x *= 2;
    x *= 3;
}
0 references
static void Bar()
{
    x *= 5;
}
```

Atomičnost

Koje sve vrednosti `x` može imati nakon pozivanja obe funkcije ako se operacija `*=` deli na: čitanje, množenje i upis i možemo se preključiti u bilo kom trenutku?

```
static int x = 1;
0 references
static void Foo()
{
    x *= 2;
    x *= 3;
}
0 references
static void Bar()
{
    x *= 5;
}
```

Odgovor: 5, 6, 10, 15 i 30.

- Težiti atomičnosti ukoliko je to moguće, da se kritične transakcije izvršavaju zajedno!
- Da li je čitanje 128-bitne `int` vrednosti atomično na 64-bitnoj arhitekturi?
- **Interlock** klasa u C#-u garantuje atomičnost!

Thread-safety

- Tip podataka ili funkcija je *thread-safe* kada se ponaša valjano i kada se poziva iz više niti, bez potrebe da implementiramo dodatnu koordinaciju.
- API može biti delimično *thread-safe* ukoliko su samo njegovi delovi *thread-safe*.
- Postizanje *thread-safety*-ja:
 - ❶ *Confinment* - ne deliti promenljive/objekte između niti (bez statičkih i globalnih promenljivih). Primer: računanje sume niza u dve niti tako što niti dobiju različite delove niza. **Nikad ne vraćati referencu na objekat nego kopiju!**
 - ❷ *Immutability* - ne menjati podatke, bez *write*-a. Primer: *const/readonly*.
 - ❸ Korišćenje *thread-safe* tipova, npr. konkurentni rečnik. Uvek čitati dokumentaciju, možda nije čitav API *thread-safe*!
 - ❹ Sinhronizacija - pristup kritičnim sekcijama omogućiti samo jednoj niti u jednom trenutku.

Da li je thread-safe?

```
public class StudentList
{
    private readonly List<string> _studentNames = new List<string>();
    private readonly object _lock = new object();
    0 references
    public void AddStudent(string studentName)
    {
        lock (_lock)
        {
            _studentNames.Add(studentName);
        }
    }
    0 references
    public void RemoveStudent(int index)
    {
        lock (_lock)
        {
            _studentNames.RemoveAt(index);
        }
    }
    0 references
    public int Size()
    {
        lock (_lock)
        {
            return _studentNames.Count;
        }
    }
    0 references
    public string GetStudentName(int index)
    {
        lock (_lock)
        {
            return _studentNames[index];
        }
    }
}
```

Da li je thread-safe?

```
public class StudentList
{
    private readonly List<string> _studentNames = new List<string>();
    private readonly object _lock = new object();
    0 references
    public void AddStudent(string studentName)
    {
        lock (_lock)
        {
            _studentNames.Add(studentName);
        }
    }
    0 references
    public void RemoveStudent(int index)
    {
        lock (_lock)
        {
            _studentNames.RemoveAt(index);
        }
    }
    0 references
    public int Size()
    {
        lock (_lock)
        {
            return _studentNames.Count;
        }
    }
    0 references
    public string GetStudentName(int index)
    {
        lock (_lock)
        {
            return _studentNames[index];
        }
    }
}
```

Metode jesu *thread-safe* ali *API* kao celina nije.

Da li je thread-safe?

```
public class StudentList
{
    private List<Student> _students;

    0 references
    public StudentList(List<Student> students)
    {
        _students = students;
    }

    0 references
    public int Size()
    {
        return _students.Count;
    }

    0 references
    public string GetStudentName(int index)
    {
        return _students[index].Name;
    }

    0 references
    public void SetStudentName(int index, string newName)
    {
        _students[index].Name = newName;
    }
}
```

Da li je thread-safe?

```
public class StudentList
{
    private List<Student> _students;

    0 references
    public StudentList(List<Student> students)
    {
        _students = students;
    }

    0 references
    public int Size()
    {
        return _students.Count;
    }

    0 references
    public string GetStudentName(int index)
    {
        return _students[index].Name;
    }

    0 references
    public void SetStudentName(int index, string newName)
    {
        _students[index].Name = newName;
    }
}
```

Ako je *Student* thread-safe tip onda jeste, u suprotnom nije.

Da li je thread-safe?

```
public class StudentList
{
    private List<string> _studentNames;

    0 references
    public StudentList(List<string> studentNames)
    {
        _studentNames = studentNames;
    }

    0 references
    public int Size()
    {
        return _studentNames.Count;
    }

    0 references
    public string GetStudentName(int index)
    {
        return _studentNames[index];
    }

    0 references
    public int GetStudentIndex(string nameToFind)
    {
        for (int i = 0; i < _studentNames.Count; i++)
        {
            if (_studentNames[i] == nameToFind)
            {
                return i;
            }
        }
        return -1;
    }
}
```

Da li je thread-safe?

```
public class StudentList
{
    private List<string> _studentNames;

    0 references
    public StudentList(List<string> studentNames)
    {
        _studentNames = studentNames;
    }

    0 references
    public int Size()
    {
        return _studentNames.Count;
    }

    0 references
    public string GetStudentName(int index)
    {
        return _studentNames[index];
    }

    0 references
    public int GetStudentIndex(string nameToFind)
    {
        for (int i = 0; i < _studentNames.Count; i++)
        {
            if (_studentNames[i] == nameToFind)
            {
                return i;
            }
        }
        return -1;
    }
}
```

Da, ne menjaju se vrednosti (*immutability*).

Da li je thread-safe?

```
public class StudentRegistry
{
    private Dictionary<string, Student> _students = new Dictionary<string, Student>();
    private readonly object _lock = new object();

    0 references
    public Student GetStudent(string key)
    {
        lock (_lock)
        {
            return _students[key];
        }
    }

    0 references
    public void SetStudent(string key, Student student)
    {
        lock (_lock)
        {
            _students[key] = student;
        }
    }
}
```

Da li je thread-safe?

```
public class StudentRegistry
{
    private Dictionary<string, Student> _students = new Dictionary<string, Student>();
    private readonly object _lock = new object();

    0 references
    public Student GetStudent(string key)
    {
        lock (_lock)
        {
            return _students[key];
        }
    }

    0 references
    public void SetStudent(string key, Student student)
    {
        lock (_lock)
        {
            _students[key] = student;
        }
    }
}
```

Ne, vraća se referenca pa dolazi do curenja interne reprezentacije.

Da li je thread-safe?

```
public class StudentRegistry
{
    private readonly Dictionary<string, string> _students = new Dictionary<string, string>();
    private readonly object _lock = new object();

    0 references
    public string GetStudent(string key)
    {
        lock (_lock)
        {
            return _students[key];
        }
    }

    0 references
    public void SetStudent(string key, string name)
    {
        lock (_lock)
        {
            _students[key] = name;
        }
    }
}
```

Da li je thread-safe?

```
public class StudentRegistry
{
    private readonly Dictionary<string, string> _students = new Dictionary<string, string>();
    private readonly object _lock = new object();

    0 references
    public string GetStudent(string key)
    {
        lock (_lock)
        {
            return _students[key];
        }
    }

    0 references
    public void SetStudent(string key, string name)
    {
        lock (_lock)
        {
            _students[key] = name;
        }
    }
}
```

Da, string je *reference type* ali je *immutable*, neće se izmeniti njegov sadržaj.

nastaviće se...